

Структура и йерархия на паметта

Ивайло Андреев + gemini 3 Flash

31 май 2026

Съдържание

1	Йерархия на паметта - кеш-памет, оперативна памет и виртуална памет	2
1.1	Йерархична структура на паметта	2
1.2	Виртуална памет	3
2	Сегментна преадресация	3
2.1	Сегментен селектор	3
2.2	Сегментен дескриптор	3
2.3	Сегментни таблици и регистри	4
2.3.1	Процес на преадресация	4
3	Странична преадресация	4
3.1	Каталог на страниците (Page Directory)	5
3.2	Описател на страница (Page Table Entry - PTE)	5
3.3	Стратегии на подмяна на страниците	5
4	Система за прекъсване - приоритети и обслужване	6
4.1	Структура и обработка	6
4.2	Типове прекъсвания	7
4.3	Конкурентност и приоритети	7
4.4	Контролери на прекъсванията	7

1 Йерархия на паметта - кеш-памет, оперативна памет и виртуална памет

Паметта в компютърните системи служи за съхранение на данни и инструкции, необходими за работата на програмите. На физическо ниво тя е изградена от битове, които се групират в байтове (8 бита), явяващи се основната адресируема единица. Тъй като различните технологии за производство на памет се различават по цена, скорост и капацитет, в съвременните компютри се използва строго дефинирана йерархична структура.

1.1 Йерархична структура на паметта

Йерархията на паметта се основава на принципа, че колкото по-близо се намира дадена памет до ядрото на централния процесор (ЦП), толкова по-бърза е тя, но и с по-малък капацитет и по-висока цена на бит. Основните нива са:

1. **Регистри на процесора:** Намират се директно в процесорното ядро, работят със скоростта на процесора, но имат пренебрежимо малък капацитет.
2. **Кеш-памет (Cache memory):** Кеш паметта обикновено се реализира чрез SRAM. Разделя се на нива (L1, L2, L3). Служи за междинен буфер между ЦП и основната памет, съхранявайки най-често използваните данни.
3. **Оперативна (основна) памет (ОП / RAM):** Изградена от динамична памет (DRAM). Тя е енергозависима (губи съдържанието си при липса на захранване) и пази текущо изпълняваните програми.
4. **Външна памет (SSD/HDD):** Енергонезависима памет с голям капацитет и ниска цена, но с много по-ниска скорост на достъп.
5. **Виртуална памет:** Логически механизъм за управление на адресното пространство, реализиран чрез странична организация и евентуално използване на дисково пространство.

Ефективността на тази йерархия се дължи на **принципа на локалност**, който бива два вида:

- **Времева (темпорална) локалност:** Ако дадена клетка от паметта е била адресирана веднъж, съществува голяма вероятност тя да бъде адресирана отново в близко бъдеще (например при цикли).
- **Пространствена локалност:** Ако дадена клетка е адресирана, съществува голяма вероятност съседните на нея клетки също да бъдат адресирани скоро (например при последователно обхождане на масиви).

За да се оптимизира трансферът, данните между кеша и оперативната памет се обменят на фиксирани блокове, наречени **кеш линии** (обикновено по 64 байта).

Когато търсените от ЦП данни са в кеша, се отчита събитие *Cache Hit* (попадение), а когато липсват – *Cache Miss* (пропуск), което налага извличането им от по-долното бавно ниво.

1.2 Виртуална памет

Виртуалната памет е софтуерно-хардуерна технология, която позволява на операционната система (ОС) да предостави на дадена програма илюзията за непрекъснато адресно пространство, което може да надвишава реалния физически обем на инсталираната RAM памет. Част от твърдия диск или SSD се заделя под формата на **свап файл (swap/page file)**, където се записват пасивните процеси с цел освобождаване на физическа RAM за активните задачи.

2 Сегментна преадресация

Сегментната преадресация е класически механизъм за управление на паметта в защитен режим (*Protected Mode*), при който логическото адресно пространство се разделя на части с променлива дължина, наречени **сегменти** (например сегмент за код, сегмент за данни, сегмент за стек). Този модел осигурява защита на програмите и ОС от неволно или злонамерено взаимно модифициране. В съвременните 64-битови операционни системи ролята на сегментацията е силно ограничена и основният механизъм за защита е страничната преадресация.

При сегментния модел логическият (виртуален) адрес се състои от две части: **сегментен селектор** и **отместване (offset)**.

2.1 Сегментен селектор

Сегментният селектор е 16-битова структура, която се зарежда в сегментните регистри на процесора (CS, DS, SS, ES, FS, GS) и съдържа следните полета:

- **Index (битове 3-15)**: Указва точния номер на записа (дескриптора) в съответната сегментна таблица. Тъй като полето е 13 бита, таблицата може да съдържа до $2^{13} = 8192$ дескриптора.
- **TI (Table Indicator - бит 2)**: Определя коя таблица се използва – 0 за Глобалната дескрипторна таблица (GDT) и 1 за Локалната дескрипторна таблица (LDT).
- **RPL (Requested Privilege Level - битове 0-1)**: Определя нивата на привилегии за достъп (от 0 за най-високо привилегированя Ring 0 / ядрото на ОС, до 3 за Ring 3 / потребителските програми).

2.2 Сегментен дескриптор

Сегментният дескриптор е 8-байтов (64-битов) запис в дескрипторната таблица, който описва характеристиките на конкретния сегмент:

- **Базов адрес (Base Address):** При 32-битовия x86 базовият адрес е 32-битов и указва началния физически адрес в паметта, от който започва сегментът.
- **Граница (Limit):** 20-битово поле, определящо максималния размер на сегмента. Заедно с бита за гранулираност (G), границата може да дефинира сегмент с максимален размер до 4 GB.
- **Атрибути за достъп и защита:** Битове, оказващи дали сегментът е само за четене, за изпълнение, какво е необходимото ниво на привилегии (DPL) и дали сегментът е зареден във физическата памет (P - Present бит).

2.3 Сегментни таблици и регистри

Таблиците съхраняват дескрипторите и се намират в оперативната памет. Процесорът разполага със специални вътрешни регистри, които съдържат физическия базов адрес и размера на самите дескрипторни таблици в ОП:

- **GDTR:** Пази адреса на Глобалната дескрипторна таблица (GDT), споделена от всички процеси.
- **LDTR:** Пази адреса на Локалната дескрипторна таблица (LDT), специфична за конкретен процес.

2.3.1 Процес на преадресация

Хардуерното устройство MMU (*Memory Management Unit*) извлича индекса от сегментния селектор, намира съответния дескриптор в таблицата (сочена от GDTR/LDTR), проверява правата за достъп и дали отместването не надвишава дефинираната граница. Ако всичко е валидно, **линейният адрес** се изчислява по формулата:

$$\text{Линеен адрес} = \text{Базов адрес на сегмента} + \text{Отместване (Offset)}$$

3 Странична преадресация

При страничната преадресация линейното адресно пространство се разделя на блокове с фиксиран размер, наречени **виртуални страници**, а физическата памет – на съответстващи блокове със същия размер, наречени **физически страници (рамки / page frames)**. Стандартният размер на една страница в x86 архитектурата е 4 KB (2^{12} байта).

Линейният адрес при класическата 32-битова x86 двунивова странична организация се разделя на три компонента: индекс за каталог на страниците, индекс за таблица на страниците и отместване в страницата (offset).

3.1 Каталог на страниците (Page Directory)

Каталогът на страниците е началното (първото) ниво в йерархията на страничното организиране. Процесорът намира неговия базов физически адрес чрез специализирания контролен регистър **CR3**. Каталогът съдържа 1024 записа (Page Directory Entries - PDE), всеки от които е с размер 4 байта. Всеки запис в каталога сочи към физическия адрес на конкретна таблица на страниците от второто ниво.

3.2 Описател на страница (Page Table Entry - PTE)

Второто ниво се състои от самите **Таблицы на страниците**, всяка от които също съдържа 1024 описателя (записа) на страници. Описателят е 32-битова структура, която съдържа:

- **Базов адрес на рамката:** Горните 20 бита сочат към началния физически адрес на 4-килобайтовата рамка в реалната RAM.
- **P (Present) бит:** Указва дали страницата се намира физически в RAM паметта (1) или е изнесена на диска (0). Ако $P = 0$, при опит за достъп процесорът генерира изключение, наречено **Page Fault** (страничен пропуск), което кара ОС да осигури страницата и да обнови таблиците на страниците.
- **R/W (Read/Write) бит:** Определя дали страницата е достъпна само за четене или и за запис.
- **U/S (User/Supervisor) бит:** Определя дали страницата е достъпна за потребителски приложения (Ring 3) или само за ядрото (Ring 0).
- **D (Dirty) бит:** Вдига се автоматично в 1, ако в страницата е правено записване на данни, което указва, че тя трябва да бъде обновена на диска преди подмяна.

3.3 Стратегии на подмяна на страниците

Когато възникне *Page Fault* и физическата RAM е напълно запълнена, операционната система трябва да избере коя съществуваща страница да премахне (дезактивира), за да освободи място за новата. За целта се използват следните алгоритми:

- **FIFO (First-In, First-Out):** Първата заредена страница в паметта се изхвърля първа. Алгоритъмът е лесен за имплементация, но неефективен, тъй като може да премахне често използвани базови страници. При него се наблюдава *аномалията на Белад* – увеличаването на броя на физическите рамки може да доведе до увеличаване на броя на страничните пропуски.
- **LRU (Least Recently Used):** Изхвърля се страницата, която не е била адресирана най-дълго време. Методът се базира на времевата локалност и

обикновено осигурява висока ефективност, но изисква сериозна софтуерно-хардуерна поддръжка (поддържане на времеви марки или броячи).

- **NRU (Not Recently Used) / Clock (Часовников алгоритъм):** Приближение на LRU, използващо бита за достъп (A - Accessed) в PTE. Страниците са организирани в кръгов списък. Указател (стрелка) обхожда страниците; ако битът $A = 1$, той се нулира и стрелката преминава напред. Ако се срещне страница с $A = 0$, тя се избира за подмяна.

За драстично ускоряване на преадресацията, MMU използва специализиран хардуерен кеш в процесора, наречен **TLB** (*Translation Lookaside Buffer*). TLB представя асоциативна кеш памет, която съхранява последните виртуално-физически трансляции. При **TLB Hit** преадресацията се извършва незабавно, а при **TLB Miss** се извършва обход на таблиците на страниците.

4 Система за прекъсване - приоритети и обслужване

Системата за прекъсване е съвкупност от хардуерни и софтуерни средства, които позволяват на процесора временно да преустанови изпълнението на текущата програма, да реагира на възникнало събитие и след това да се завърне към прекъснатата нишка без загуба на контекст.

4.1 Структура и обработка

Когато възникне събитие, изискващо прекъсване, се активира следният базов алгоритъм:

1. Завършва се изпълнението на текущата машинна инструкция.
2. Хардуерът автоматично записва в системния стек минималния текущ контекст на процесора: флаговия регистър (*EFLAGS*), кодовия сегмент (*CS*) и програмния брояч (*EIP*).
3. Процесорът извлича номер на прекъсване (вектор) и по него намира съответния дескриптор в **Таблицата на векторите за прекъсване (IDT)**.
4. Зарежда се адресът на съответната обслужваща подпрограма – **ISR** (*Interrupt Service Routine*).
5. Изпълнява се ISR, като в нейния край задължително се поставя специалната машинна инструкция **IRET** (*Interrupt Return*).
6. Инструкцията IRET възстановява автоматично от стека стойностите на *EFLAGS*, *CS* и *EIP* и прекъснатата програма продължава работа.

4.2 Типове прекъсвания

Прекъсванията се разделят на три основни категории:

1. **Хардуерни (външни):** Генерират се от периферни устройства посредством електрически сигнали към пиновете на процесора (напр. натискане на клавиш, пристигане на мрежов пакет). Биват два подтипа:
 - *Маскируеми:* Могат да бъдат временно игнорирани от процесора, ако се нулира флагът за прекъсване ($IF = 0$) чрез инструкцията CLI.
 - *Немаскируеми (NMI):* Подават се по специална хардуерна линия и не могат да бъдат забранени. Използват се за критични събития (например хардуерна повреда или спиране на захранването).
2. **Вътрешни (Изключения / Exceptions):** Генерират се синхронно от самия процесор при възникване на грешка по време на изпълнение на дадена инструкция (например делене на нула, невалиден операционен код, *Page Fault*).
3. **Софтуерни:** Предизвикват се умишлено от програмиста чрез изпълнение на изрична инструкция (например INT n или SYSENTER при x86 архитектури). Използват се за реализиране на системни извиквания (*System Calls*) към операционната система.

4.3 Конкурентност и приоритети

В компютърните системи множество устройства могат да заявят прекъсване едновременно. За разрешаване на конфликтите се въвежда система от **приоритети**.

- Приоритетът се определя от архитектурата и контролера на прекъсванията.
- **Вложени прекъсвания:** Текущо изпълнявана подпрограма за обслужване на прекъсване (ISR) може да бъде прекъсната единствено от нова заявка, която притежава по-висок приоритет от нейния. Заявки с по-нисък или равен приоритет се задържат на опашка до приключване на текущото обслужване.

Общият ред на обслужване при едновременно възникване на заявки е: Софтуерни прекъсвания и изключения → Прекъсване при стъпков режим (Debug) → Немаскируеми прекъсвания (NMI) → Маскируеми хардуерни прекъсвания.

4.4 Контролери на прекъсванията

Тъй като процесорът притежава ограничен брой физически пинове за приемане на прекъсвания, управлението на външните заявки се поема от специализирано хардуерно устройство – **контролер на прекъсванията** (например класическия чип *8259A PIC* или съвременния *APIC*).

Контролерът приема индивидуалните заявки от устройствата през линиите **IRQ** (*Interrupt Request*), анализира техните приоритети, решава конфликтите при конкурентни заявки и подава сигнал към единствения пин за маскируеми прекъсвания на ЦП, изпращайки по шината съответния номер на вектор.